

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1003	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	V.Tejaswini	Reg. No.:	192424360
Date:	17/08/2026	Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a Git & Docker Technical Assignment by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. Problem Analysis

Question

Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

Answer:

The **DevOps-Based Automated Competitive Exam Preparation and Learning Management Platform** is developed to provide students with a centralized platform for competitive examination preparation. The system allows students to access study materials, practice questions, mock tests, performance reports, and personalized learning recommendations. DevOps practices are integrated into the project to improve development, version control, testing, and deployment.

The main objective is to create an efficient, reliable, and user-friendly learning platform that supports students throughout their exam preparation. The platform also uses Git for source-code management and Docker for application containerization and deployment.

Project Objectives:

- Provide a centralized competitive exam preparation platform.
- Provide study materials and practice questions.
- Conduct online quizzes and mock examinations.
- Automatically evaluate student answers.
- Track student performance and progress.
- Provide AI-powered learning recommendations.
- Improve software development using DevOps practices.
- Manage source code using Git.
- Deploy the application using Docker.

Functional Requirements:

- Student registration and login.
- User profile management.
- Subject and topic management.
- Study material management.
- Online quizzes and mock examinations.
- Automatic answer evaluation.
- Score calculation.
- Performance tracking.
- AI-based recommendations.
- Progress dashboard.
- Admin management.
- Database management.

Expected Outcomes:

The expected outcome is a reliable and user-friendly platform where students can prepare for competitive examinations, attend mock tests, analyze their performance, and receive personalized learning recommendations. Git and Docker improve the development and deployment process.

Question

Identify the project objectives, functional requirements, and expected outcomes.

Answer:

The project objectives are to provide an organized learning environment, automate examination activities, analyze student performance, and provide personalized learning support.

The major functional requirements include student authentication, study material access, online examinations, automatic evaluation, performance analysis, AI-based recommendations, and administrative management.

The expected outcome is a complete competitive exam preparation platform that provides efficient learning, assessment, performance tracking, and personalized recommendations.

2. Project Structure Design

Question 2(a):

Design and explain the folder structure of your project repository.

Answer:

The project repository is organized into separate folders for frontend, backend, database, testing, documentation, and Docker configuration.

competitive-exam-platform/

```
|
|
|— backend/
|   |— app.py
|   |— models/
|   |— routes/
|   |— services/
|   └— requirements.txt
|
|
|— frontend/
|   |— src/
|   |— public/
|   └— package.json
|
|
|— database/
|   └— schema.sql
|
|
|— tests/
|   |— test_auth.py
|   |— test_exam.py
|   └— test_performance.py
|
|
|— docs/
```

```
| └─ project_documentation.md
|
├─ Dockerfile
├─ docker-compose.yml
├─ .gitignore
└─ README.md
```

This structure separates different components of the application and makes the project easier to develop, test, maintain, and deploy.

Question

Justify the purpose of each major folder and file included in the repository.

Answer:

backend/ – Contains the server-side code, APIs, business logic, and database operations.

frontend/ – Contains the user interface and client-side application files.

database/ – Contains database schemas and database-related scripts.

tests/ – Contains automated test cases used to verify application functionality.

docs/ – Stores project documentation and technical information.

Dockerfile – Contains instructions for building the Docker image.

docker-compose.yml – Defines and manages multiple application containers.

.gitignore – Specifies files that should not be tracked by Git.

README.md – Provides project description, installation instructions, commands, and usage details.

requirements.txt – Contains the Python dependencies required by the backend application.

3. Git Repository Initialization

Question 3(a):

Create a Git repository for the project.

Answer:

A Git repository is created to track and manage all source-code changes.

First, create the project directory:

```
mkdir competitive-exam-platform
```

```
cd competitive-exam-platform
```

Initialize the Git repository:

```
git init
```

Add the project files:

```
git add .
```

Create the initial commit:

```
git commit -m "Initial project setup"
```

The local repository can then be connected to GitHub:

```
git remote add origin <repository-url>
```

Finally, push the project:

```
git push -u origin main
```

Question

Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

Answer:

The main steps involved in Git repository initialization are creating the project directory, initializing Git using `git init`, adding files using `git add`, creating a commit using `git commit`, connecting the remote repository, and pushing the project to GitHub.

Essential Git Files:

.git/ – Stores Git's internal repository information, commits, branches, and configuration.

.gitignore – Prevents unnecessary and sensitive files from being tracked.

README.md – Provides information about the project and instructions for using it.

Git initialization provides a controlled environment for tracking and managing project development.

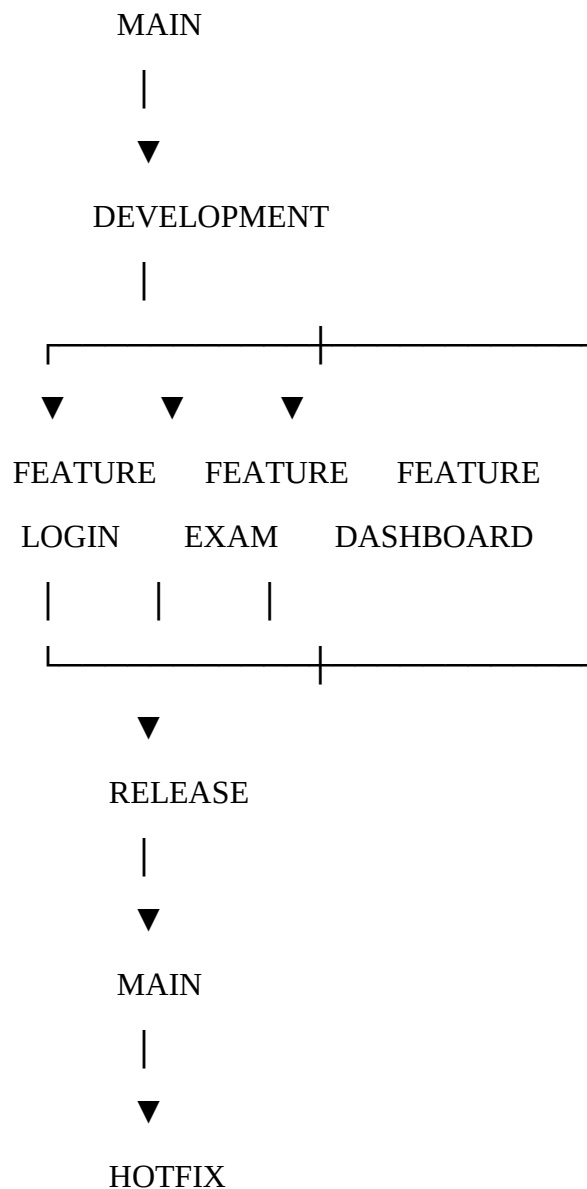
4. Git Branching Strategy

Question

Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).

Answer:

A structured branching strategy is used for the project.



Main: Contains stable and production-ready code.

Development: Contains integrated features under testing.

Feature: Used for developing individual features such as login, exams, and dashboards.

Release: Used for final testing before production deployment.

Hotfix: Used to fix critical production issues.

Justify why the selected branching model is appropriate for the assigned project.

Answer:

The selected branching model is appropriate because the project contains multiple modules that can be developed independently. Feature branches allow developers to work on separate modules without affecting stable code. The development branch allows integration and testing before the code reaches the main branch.

This strategy reduces conflicts, improves collaboration, keeps production code stable, and provides better control over software releases.

5. Version Control Workflow

Question

Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.

Answer:

The Git workflow is performed using the following commands.

Create a Feature Branch:

```
git checkout -b feature/login
```

Add Changes:

```
git add .
```

Commit Changes:

```
git commit -m "Add user login functionality"
```

Push the Branch:

```
git push -u origin feature/login
```

Merge the Feature:

```
git checkout development
```

```
git merge feature/login
```

Synchronize the Repository:

```
git pull origin development
```

```
git push origin development
```


Resolve Conflicts:

If a merge conflict occurs, the conflicting file is manually corrected and then committed:

```
git add .
```

```
git commit -m "Resolve merge conflict"
```

Explain the purpose of each Git operation performed.**Answer:**

git status – Checks the current state of the repository.

git checkout -b – Creates and switches to a new branch.

git add – Adds modified files to the staging area.

git commit – Records changes in the Git history.

git push – Uploads local changes to the remote repository.

git pull – Downloads and integrates changes from the remote repository.

git merge – Combines changes from one branch into another.

Conflict resolution – Corrects differences when Git cannot automatically merge changes.

These operations provide controlled version management and support effective collaboration.

6. Commit History Analysis**Question**

Present the commit history of the project.

Answer:

The project commit history can be viewed using:

```
git log --oneline
```

Example commit history:

Initial project setup

Add project folder structure

Implement user authentication

Add online examination module

Add performance dashboard

Add database configuration

Add automated tests

Add Dockerfile

Add Docker Compose configuration

Fix deployment configuration

The commit history shows the development progress of the project and provides a record of important changes.

Explain how commit messages follow good version control practices and contribute to project maintenance.

Answer:

Good commit messages are short, clear, meaningful, and related to a specific change. Examples include:

Add user authentication

Implement examination module

Fix database connection

Add performance dashboard

Add Docker deployment

Meaningful commit messages help developers understand the project history. They make debugging easier, help identify when a particular change was introduced, and allow unwanted changes to be reverted easily.

7. Docker Environment Design

Question

Explain why Docker is suitable for the assigned project.

Answer:

Docker is suitable because it packages the application and its dependencies into isolated containers. It provides the same execution environment during development, testing, and deployment.

Docker improves portability, reduces environment-related errors, simplifies deployment, and makes it easier to scale application components. It also allows frontend, backend, and database services to run independently.

Question 7(b):

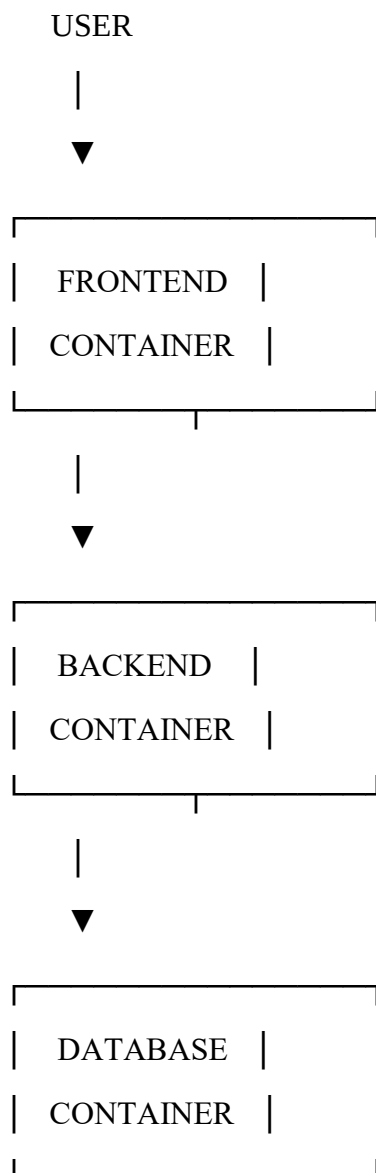
Identify the application components that require containerization.

Answer:

The main application components requiring containerization are:

1. **Frontend Container** – Runs the user interface.
2. **Backend Container** – Runs the application server and APIs.
3. **Database Container** – Stores user, question, examination, and performance data.

Docker Architecture:



8. Dockerfile Development

Question

Design and explain the Dockerfile required to containerize the application.

Answer:

A Dockerfile is created to define the environment required to run the backend application.

```
FROM python:3.12-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

The Dockerfile creates a reproducible environment containing the required Python runtime, dependencies, application files, and startup command.

Describe the purpose of each instruction used in the Dockerfile.

Answer:

FROM – Selects the base Python image.

WORKDIR – Sets the working directory inside the container.

COPY – Copies files from the project into the container.

RUN – Installs the required dependencies.

EXPOSE – Indicates the port used by the application.

CMD – Defines the command executed when the container starts.

These instructions together create a Docker image that can run the application consistently.

9. Docker Compose Design

Question

Develop a Docker Compose configuration for the project.

Answer:

The following Docker Compose configuration defines the frontend, backend, and database services.

services:

backend:

build: ./backend

ports:

- "5000:5000"

environment:

DATABASE_URL: postgresql://examuser:password@database:5432/examdb

depends_on:

- database

frontend:

build: ./frontend

ports:

- "3000:3000"

depends_on:

- backend

database:

image: postgres:16

environment:

POSTGRES_DB: examdb

POSTGRES_USER: examuser

POSTGRES_PASSWORD: password

volumes:

- postgres_data:/var/lib/postgresql/data

volumes:

postgres_data:

Explain the services, networking, volumes, environment variables, and dependencies used.

Answer:

Services: The configuration contains three services: frontend, backend, and database.

Networking: Docker Compose creates a common network that allows the services to communicate with each other.

Volumes: The PostgreSQL volume stores database information permanently.

Environment Variables: Database name, username, password, and connection details are provided through environment variables.

Dependencies: depends_on specifies the order in which dependent services are started.

This configuration simplifies the management of the complete application.

10. Container Deployment and Testing

Question

Demonstrate the execution of the application using Docker containers.

Answer:

The application is built using:

docker compose build

The containers are started using:

docker compose up -d

Running containers are checked using:

docker ps

Application logs are viewed using:

docker compose logs

The frontend can be accessed at:

http://localhost:3000

The backend can be accessed at:

`http://localhost:5000`

The containers can be stopped using:

`docker compose down`

Explain the testing procedure and provide evidence that the application runs successfully.

Answer:

The application is tested after starting the Docker containers. First, the running containers are verified using `docker ps`. The frontend is opened in a browser and the backend APIs are tested. User registration and login are verified, followed by examination functionality, automatic evaluation, and performance tracking.

Database communication is also tested to ensure that student information, questions, and results are stored correctly.

Evidence:

The following screenshots can be included in the report:

- `docker compose build` execution.
- `docker compose up -d` execution.
- `docker ps` showing running containers.
- Docker logs.
- Application running in the browser.
- Git branches.
- Git commit history.
- Git merge operation.

Successful completion of these tests confirms that the application is running correctly inside Docker containers.

11. Benefits of Git and Docker

Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.

Answer:

Git improves collaboration by allowing multiple developers to work on different branches. It maintains the complete history of project changes and allows previous versions to be restored.

Docker improves portability by packaging the application with its dependencies. It provides a consistent environment across different systems and simplifies application deployment.

Docker also supports scalability because additional containers can be created when application usage increases. Containerization provides isolation between application components and makes maintenance easier.

Together, Git and Docker provide an efficient DevOps workflow that improves collaboration, version management, portability, deployment, scalability, and maintainability.

Explain the major benefits of using Git and Docker in the project.

Answer:

Git Benefits:

- Version control.
- Team collaboration.
- Branch management.
- Change tracking.
- Easy rollback.
- Better project maintenance.

Docker Benefits:

- Application portability.
- Environment consistency.
- Fast deployment.
- Container isolation.
- Easy scaling.
- Dependency management.

The combined use of Git and Docker makes the application development and deployment process more reliable and efficient.

12. Challenges and Improvements

Question

Identify the challenges encountered during implementation.

Answer:

The major challenges encountered during implementation include:

- Managing multiple Git branches.
- Synchronizing local and remote repositories.
- Resolving Git merge conflicts.
- Understanding Docker images and containers.
- Configuring frontend, backend, and database communication.
- Managing environment variables.
- Installing dependencies inside containers.
- Debugging application errors using Docker logs.
- Maintaining persistent database storage.
- Ensuring consistent application execution across environments.

These challenges were handled through proper Git workflows, Docker configuration, testing, and debugging.

Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Answer:

The Git workflow can be improved by implementing pull requests, code reviews, automated testing, and CI/CD pipelines. GitHub Actions can be used to automatically build and test the application whenever new changes are pushed.

Docker deployment can be improved by using multi-stage Docker builds, health checks, secure secret management, image versioning, monitoring, and automated security scanning.

Future Enhancements:

- Implement CI/CD using GitHub Actions.
- Add automated unit and integration testing.
- Use Docker multi-stage builds.
- Implement secure secret management.

- Add container health checks.
- Add monitoring and centralized logging.
- Use cloud deployment.
- Use Kubernetes for large-scale deployment.
- Add advanced AI-powered learning recommendations.
- Implement adaptive examinations and real-time analytics.

These improvements can make the platform more secure, scalable, reliable, and suitable for a large number of users.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution,	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.

	and professional commit messages with clear history.			
Docker Implementation(20 %)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15 %)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

	improvements			
	.			

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25